

Smart Health & Fitness Tracker — Application Testing Summary

Module: COM661 Full Stack Strategies and Development **Assignment:** CW2 (Front-End)

1. Testing Strategy

Two complementary testing approaches were used during the development of the front-end application:

Automated unit tests — written using *Jasmine* as the assertion framework and *Karma* as the test-runner (Angular's default toolchain). These tests focus on the parts of the application where regressions are most likely: services that talk to the API, the JWT authentication store, the route guards, and the form validation logic in critical components.

Manual functional testing — every page was exercised end-to-end against a locally-running CW1 Flask API (with the database seeded via `seed_data.py`) using the supplied admin and user accounts. A structured test plan was followed and every user-facing path was verified for a happy path and at least one error path. Postman was also used to confirm that the requests issued by the front-end matched the expected shape.

2. Tooling

Tool	Version	Purpose
Jasmine	5.1.x	Test assertions / spec definition syntax
Karma	6.4.x	Cross-browser test runner; spawns Chrome (headless in CI)
Angular HttpClientTestingModule	17.3.x	Stubs out HTTP calls so service tests run without a server
TestBed	17.3.x	Angular DI configuration for component/service tests
Postman	11.x	Manual verification of request payloads and response codes
Browser DevTools	Chrome 124+	Network tab inspection + Lighthouse accessibility audit

3. Automated Unit Tests

The following spec files are included in the submission and can be executed with `npm test` (interactive Karma) or `npm run test:ci` (single-run, headless Chrome):

Spec file	Subject	Cases	What is verified
auth.service.spec.ts	AuthService	5	Construction; initial logged-out state; persists token + user on successful login; recognises admin role; clears state and navigates on logout; updateLocalUser merges patches into stored profile.
activity.service.spec.ts	ActivityService	5	GET with pagination params; GET with filter params (activity_type, start_date, end_date); POST sends body verbatim; PUT to /:id; DELETE to /:id. HttpClientTestingModule asserts URL, method and querystring.
toast.service.spec.ts	ToastService	5	Empty initial queue; success() pushes a toast of correct kind; consecutive toasts get unique IDs; auto-dismiss after 4 seconds; explicit dismiss(id) removes the targeted toast immediately.
auth.guard.spec.ts	authGuard	2	Unauthenticated request returns false and redirects to /login with the original URL preserved as a redirect query parameter; authenticated request returns true.

login.component.spec.ts	LoginComponent	7	Component construction; empty form is invalid; invalid email is flagged; short password is flagged; valid credentials accepted; fillDemo('user') and fillDemo('admin') populate correct credentials; password visibility toggles correctly.
-------------------------	----------------	---	---

Total: 5 spec files, 24 individual test cases, all passing on the submitted code base.

4. Manual Test Matrix

All test cases below were executed against the seeded development environment immediately before submission. Pass = expected outcome observed; the column "Status code" refers to the HTTP response actually returned by the back-end.

#	Area	Test case	Expected	Status code	Result
1	Auth	Register a new account with valid data	Account created, JWT issued, redirected to dashboard	201	PASS
2	Auth	Register with an email already in the database	Toast: 'An account with this email already exists'	409	PASS
3	Auth	Register with an invalid email format	Client-side validator fires before submit	n/a	PASS
4	Auth	Login with correct credentials (alice@example.com)	Welcome toast, dashboard renders, JWT in localStorage	200	PASS
5	Auth	Login with wrong password	Toast: 'Invalid email or password'	401	PASS
6	Auth	Visit /dashboard while logged out	Redirected to /login?redirect=/dashboard	n/a	PASS
7	Auth	Logout then click browser Back button	authGuard intercepts; user lands on /login again	n/a	PASS
8	Profile	Edit full_name then save	Toast 'Profile updated'; navbar avatar reflects new initial	200	PASS
9	Profile	Save with no fields changed	Toast 'No changes to save'; no API call	n/a	PASS
10	Profile	Delete own account (confirm)	Account removed, auto-logout, redirected to /login	200	PASS
11	Activities	Log a running activity (45 min, 350 kcal)	Toast success; new entry appears at top of list	201	PASS
12	Activities	Log activity with type 'flying' via DevTools	Server returns validation error; toast shows it	400	PASS
13	Activities	Filter list by activity_type=running	Querystring sent; only running activities shown	200	PASS
14	Activities	Date range filter 2026-03-01 to 2026-03-31	Both dates appended as querystring; correct subset returned	200	PASS
15	Activities	Pagination: navigate to page 2 of 3	page=2 in querystring; pagination buttons disable correctly	200	PASS
16	Activities	Edit existing activity	Form pre-populated; PUT issued; list refreshes	200	PASS
17	Activities	Delete activity (confirm dialog)	Modal shows; on confirm DELETE issued; list refreshes	200	PASS
18	Activities	Delete activity (cancel dialog)	Modal closes; no API call issued	n/a	PASS
19	Nutrition	Log a breakfast meal with full macros	Macros saved; live calorie estimate (4/4/9) shown	201	PASS
20	Nutrition	Filter by meal_type=lunch	Querystring sent; only lunches shown	200	PASS
21	Nutrition	Edit a meal log	PUT updates; list refreshes	200	PASS
22	Nutrition	Delete meal	DELETE issued; list refreshes	200	PASS
23	Health	View metrics page	BMI computed by server displayed with colour-coded badge	200	PASS
24	Health	Update heart_rate to 75	PUT issued; new value displayed; updated_at refreshed	200	PASS
25	Health	Update heart_rate to 999 via DevTools	Toast: server validation error	400	PASS

#	Area	Test case	Expected	Status code	Result
26	Goals	Set all five goal fields and save	PUT issued; current goals card now shows them	200	PASS
27	Goals	Set target_date in the past	Server-side validator blocks; toast displays error	400	PASS
28	Goals	Clear all goals (confirm)	DELETE issued; goals removed	200	PASS
29	Analytics	Open analytics page	BMI card, calorie balance, line chart, doughnut, stacked-bar all render	200×6	PASS
30	Analytics	Adjust calorie balance date range	New start_date/end_date sent; metrics refresh	200	PASS
31	Admin	Visit /admin/users as a non-admin	adminGuard intercepts; toast 'Admin access required'	n/a	PASS
32	Admin	Search users for 'alice'	search querystring sent; only Alice returned	200	PASS
33	Admin	Filter by membership_type=premium	Querystring sent; only premium users	200	PASS
34	Admin	Promote a user to admin (confirm)	PUT /users/:id/role issued; badge updates	200	PASS
35	Admin	Delete a user (confirm)	DELETE issued; list refreshes	200	PASS
36	Admin	Try to delete own admin account from list	Blocked client-side; toast warns user to use Profile page	n/a	PASS
37	Admin	View top active users (limit=5)	GET with limit=5 sent; chart and list render	200	PASS
38	Auth	JWT expires (manually delete from localStorage)	Next API call returns 401; interceptor logs user out gracefully	401	PASS
39	Network	Stop the Flask back-end and refresh	Toast: 'Cannot reach the API server. Is the Flask back-end running?'	0	PASS
40	Responsive	Open every page at 375px viewport (iPhone SE)	Bootstrap navbar collapses; tables scroll horizontally; no overflow	n/a	PASS

Manual test pass rate: 40/40 (100%).

5. Test Coverage Commentary

The automated suite does not target 100% statement coverage. Its purpose is to lock in the behaviour of the parts of the application that are *most likely to silently break*: the AuthService (which manages JWT persistence and is shared by every request), the API service layer (where querystring assembly is easy to get wrong), the route guards (security-critical), and the most complex form (login). The remaining behaviour is verified via the manual test matrix above. This split balances test maintenance cost against catching regressions in the highest-value code paths.

6. How to Reproduce These Tests

Prerequisites. Node 18+, Chrome installed, Flask back-end running on `http://127.0.0.1:5001`, and MongoDB seeded.

Automated. `cd smart-health-frontend && npm install && npm run test:ci`

Manual. Start the front-end with `npm start`, open `http://localhost:4200/`, and walk through the matrix in section 4 using the seeded `alice@example.com / password123` and `admin@smarthealth.com / admin123` accounts.